LAPORAN PRAKTIKUM

STRUKTUR DATA

“ABSTRACT DATA TYPE”

Disusun Oleh :

Dzhillan Dzhalila

2511531001

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid A.

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan atas kehadirat Allah SWT Tuhan Yang Maha Esa yang telah memberikan rahmat dan berkatnya, sehingga penulis dapat menyelesaikan laporan praktikum Jaringan Komputer yang telah dilaksanakan sebagai pertemuan ke-1 untuk mata kuliah Praktikum Struktur Data. Praktikum ini berfokus pada pembahasan konsep adaptasi dari Abstrack Data Type pada pemorgraman bahasa Java.

Melalui praktikum ini, hasil yang diharapkan adalah penulis dapat memahami dan mengimplementasikan secara langsung konsep Abstack Data Type pda real project. Melalui projek projek sederhana yang dilakukan pada praktikum ini membantu mengembangkan pemahaman akan kebutuhan Abstrack Data Type pada program.

Penulis menyadari bahwa, laporan praktikum ini masih memiliki banyak kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan dan pengembangan di masa mendatang. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan pembelajaran penulis maupun referensi pembaca dalam memahami dasar-dasar pemrograman.

Padang, 2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada pembangunan sebuah program yang mennggunakan bahasa pemrograman, kita sebagai programmer harus mengetahui terlebih dahulu tipe data dan operasi apa saja yang kita inginkan dalam program tersebut untuk tipe data tsb. Dalam tahap pertama, tentunya kita butuh mendefinisikan atau mengklasifikan jenis nilai apa saja yang dapat disimpan oleh suatu variabel dan juga operasi apa saja yang diperbolehkan pada nilai-nilai tersebut. Agar interface program dapat digunakan secara berulang dan mudah dalam aspek penjagaan, pendefinisian ini tidak menyertakan bagaimana implementasi yang detail akan operasi terkait. Untuk pendefinisian inilah kita akan menggunakan modul tipe data yang bisa digunakan untuk semua bahasa pemrograman yang ada. Modul yang sering diimplementasikan dikenal dengan Abstrack Data Type atau ADT. Walaupun kita harus menyesuaikan metode dan logika yang berbeda untuk masing masing bahasa pemrograman, kita terap dapat melakukan semua operasi terkait yang didefinisikan untuk ADT tersebut terlepas dari jenis bahasanya.

1.2 Tujuan

1. Memahami konsep dasar Abstrack Data Type (ADT) menggunakan bahasa pemrograman Java.
2. Mengimplementasikan ADT dengan berbagai struktur data yang tersedia dalam bahasa pemrograman Java.
3. Menguasai prinsip spesifikasi pendefinisian type data dan operasi pada program.

1.3 Manfaat

1. Mengasah keterampilan pemrograman sebagai fondasi sebelum mempelajari konsep struktur data kompleks.
2. Memberikan pemahaman praktis dari implementasi ADT dalam membangun suatu program.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Abstract Data Type (ADT) adalah pendefinisian type data dan sekumpulan operasi dasar yang dapat dilakukan terhadap type data yang didefinisikan. Sekumpulan operasi dasar yang dapat dilakukan terhadap type data tersebut juga dapat disebut dengan primitif. Pada bahasa pemrograman Java spesifikasi operasi dan tipe data biasanya dikenal dengan *interface* atau *abstract class* yang akan mendefinisikan method, tipe kembalian yang digunakan.

Primitif yang digunakan dalam ADT bisa dikelompokkan menjadi :

1. Konstruktor/Kreator, yaitu pembentuk nilai type. Semua objek atau variabel harus melalui kontraktor terlebih dahulu (biasanya nama dari fungsi/ prosedur kontraktor dimulai dengan **Make**).
2. Selektor, tugasnya adalah mengakses komponen type (biasanya namanya dimulai dengan **Get**).
3. Prosedur pengubah nilai komponen (juga memakai awalan **Get**)
4. Validator komponen type, tugasnya adalah untuk melakukan test apakah dapat membentuk type sesuai dengan batasan.
5. Destructor/Dealokator, yaitu untuk "menghancurkan" nilai objek dan penyimpanannya.
6. Baca/Tulis, bertugas sebagai interface dengan input/output device.
7. Operator relasional, bertugas untuk mendefinisikan lebih besar, lebih kecil, sama dengan, dsb terhadap type tersebut.
8. Aritmatika, biasanya aritmatika dalam Bahasa pemrograman hanya terdefinisi untuk bilangan numerik.
9. Konversi dari type tersebut ke type dasar atau sebaliknya.

Semua primitif ADT ini hanya akan memberikan penjelasan mengenai apa yang bisa dioperasikan oleh suatu type data, bukan bagaimana operasi tersebut akan dilakukan. Maka dari itu, suatu program dari modul ADT membutuhkan program lain yang disebut sebagai **driver** agar operasi yang didefinisikan pada ADT bisa dilakukan pengetesan secara tuntas.

2.2 Kode Program

2.2.1 Program ADT "Jam_2511531001

```
package pekan1_2511531001;
public final class Jam_2511531001 {
	private int hh; //0..23
	private int mm; // 00..59
	private int ss; //0..59

	//Validator : mengecek apakah angkanya valid atau ga
	public static boolean isValid (int h,int m,int s) {
		return (0<=h && h<=23) && (0<=m && m<=59) && (0<= s
&& s <= 59);
	}
	//Konstruktor : mengikuti nama kelas
	public Jam_2511531001 (int h,int m, int s) {
		this.hh= h; this.mm= m; this.ss= s;
	}
	//Selektor
	public int getHour() {return hh;}
	public int getMinute() {return mm;}
	public int getSecond() {return ss;}

	//Mutator
	public void setHour(int h) {this.hh=h;}
	public void setMinute(int m) {this.mm=m;}
	public void setSecond(int s) {this.ss=s;}

	//Konversi
	public int toSeconds () {return hh*3600 + mm*60 +ss;}
	public static Jam_2511531001 fromSeconds (int total) {
		total %= 24*3600; // wrap 24 jam
		int h = total/ 3600; //total %= 3600
		int m = total / 60; int s = total & 60;
		return new Jam_2511531001 (h,m,s);
	}

	//relasional
	public int compareTo (Jam_2511531001 other) {return
Integer.compare(this.toSeconds(), other.toSeconds());}
	public boolean equals (Object o) {
		if (!(o instanceof Jam_2511531001 j)) return false;
```

```
                return hh== j.hh && mm == j.mm && ss==j.ss;
        }
        public int hashCode() {return java.util.Objects.hash(hh,mm,ss);}

        //Aritmatika
        public Jam_2511531001 plus (Jam_2511531001 other) {return
fromSeconds (this.toSeconds () + other.toSeconds());}
        public Jam_2511531001 minus (Jam_2511531001 other) {return
fromSeconds (Math.floorMod(this.toSeconds() -other.toSeconds(),
24*3600));}
        public Jam_2511531001 nextSecond() {return fromSeconds
(this.toSeconds()+1);}
        public Jam_2511531001 nextNSeconds (int n) {return fromSeconds
(this.toSeconds () + Math.max(0,n));}
        public Jam_2511531001 prevSecond() {return fromSeconds
(Math.floorMod(this.toSeconds() - 1, 24* 3600));}
        public Jam_2511531001 prevNSeconds (int n) {return fromSeconds
(Math.floorMod(this.toSeconds()- Math.max(0,n), 24*3600));}

        //Durasi (detik) : bisa negatif jika this >other? sesuai spesifikasi durasi
(JAw, JAkh)
        public static int durasiDetik (Jam_2511531001 jaw, Jam_2511531001
jakh) {return jakh.toSeconds()- jaw.toSeconds();}
        //overriding ke string
        public String toString () {return
String.format("%02d:%02d:%02d",hh,mm,ss);}

}
```

Kode Program 2.1 : Program Jam_2511531001

Program ini adalah modul ADT dari sebuah ADT JAM yang terdiri dari pendefinisian type data, dan juga primitive (Validator, Konstruktor,Mutator, Konversi, Relasional,Artimatika). Penjelasan :

1. Pendefinisian type data → dalam progam ADT ini, ada 3 variabel yang didefinsikan dan semuanya bertipe data int, yaitu hh untuk hour (jam), mm (minute) dan ss (detik). Ketiga variabel dideklarasikan dengan keyword private untuk membatasi bahwa variabel ini tidak bisa diakses melalui kelas selain kelas ketiga variabel ini berada.
2. Validator → menyatakan keadaan valid jika variabel hh itu (0-23), mm (0-59) dan ss(0-59) dan menyatakan keadaan tidak valid jika tidak memenuhi syarat tsb.

3. Selektor → mengambil nilai atribut privat dengan tidak memberikan akses dalam pengubahan nilai awal (menggunakan keyword public)
4. Mutator → mengubah nilai atribut h,m,s
5. Konversi → mengubah representasi waktu kea tau dari format detik
6. Relasional → membandingkan dua objek jam (jam ke-1 dan jam ke-2) untuk pengurutan atau pengecekan kesamaan.
7. Aritmatika → melakukan operasi aritmatika (penambahan dan pengurangan waktu) pada jam dan mengangani overflow secara otomatis.

2.2.2 Program Driver 1 "JamDriver_2511531001"

```
public class JamDriver_2511531001 {
        public static void main(String[] args) {
        Jam_2511531001 a = new Jam_2511531001 (23,59,50);
        Jam_2511531001 b = new Jam_2511531001 (0,0,15);
        System.out.println("a            = " + a);
        System.out.println("b            = " + b);
        System.out.println("a+b          = " + a.plus(b));
        System.out.println("next 20s     = " + a.nextNSeconds(20));
        System.out.println("durasi (a,b)   = " +
Jam_2511531001.durasiDetik(a,b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
        }}
```

Kode Program 2.2 : Program JamDriver_2511531001

Program diatas adalah program driver yang bertugas untuk menjadi program dimana pendefinisian operasi pada program ADT dieksekusi. Program ini dimulai dari memanggil program Jam_2511531001 dan mendeklarasikan "a" sebagai jam ke-1 dan "b" sebagai jam ke-2. Dalam pendeklarasikan program ini, nilai dari atribut jam, menit dan detik langsung diberikan.

Dan selanjutnya semua primitive yang sudah didefinisikan dipanggil untuk menghasil output sebagai berikut :

```
<terminated> JamDriver_2511531001 [Java
a                 = 23:59:50
b                 = 00:00:15
a+b               = 00:00:04
next 20s          = 00:00:08
durasi (a,b)      = -86375
a.compareTo(b)    = 1
```

Gambar 2.1 : Output Driver 1

2.2.3 Program Driver 2 “JamDriver2_2511531001”

```
import java.util.*;
public class JamDriver2_2511531001 {
        public static void main (String [] args) {
                Scanner input = new Scanner (System.in);
                System.out.println("===Program Driver Objek Jam ===");
                //1. input Jam Pertama
                System.out.println("\n--- Input Jam 1 ---");
                Jam_2511531001 j1= buatJamDariInput(input);
                //2. Input jam Kedua
                System.out.println("\n---Input Jam 2 ---");
                Jam_2511531001 j2 =buatJamDariInput(input);
                //3. Menampilkan data
                System.out.println("\n--- Hasil Operasi ---");
                System.out.println(" Jam 1 (String) : " +j1.toString());
                System.out.println(" Jam 2 (String) : " +j2.toString());
                System.out.println(" Jam 1 dalam detik : " +j1.toSeconds());
                System.out.println(" Jam 2 dalam detik : " +j2.toSeconds());
                //4. Operasi Relasional (Perbandingan)
                int perbandingan = j1.compareTo(j2);
                if (perbandingan > 0) {
                        System.out.println("Status : Jam 1 lebih lambat
(setelah) Jam 2");
                }
                else if ( perbandingan < 0) {
                        System.out.println("Status : Jam 1 lebih cepat awal
(sebelum) Jam 2");
                }
                else {
                        System.out.println("Status : Jam 1 dan Jam 2 sama
persis");
                }

                //5. Operasi Aritmatika
                System.out.println("Durasi (J1 ke J2) : " +
Jam_2511531001.durasiDetik(j1,j2)+" detik");

                Jam_2511531001 jNext = j1.nextSecond();
```

```
                System.out.println("Jam 1 Detik Berikutnya : " +jNext);

                Jam_2511531001 jPrev = j1.prevSecond();
                System.out.println("Jam 1 Detik Sebelumnya : " +jPrev);

                //6.Operasi Penjumlahan Jam
                Jam_2511531001 jHasilPlus = j1.plus(j2);
                System.out.println("Hasil J1 + J2 : " + jHasilPlus);

                input.close();
                System.out.println("\nProgram Selesai");
        }

        public static Jam_2511531001 buatJamDariInput (Scanner sc) {
                int h,m,s;
                while (true) {
                        System.out.print("Masukkan Jam (0-23) : ");
                        h=sc.nextInt();
                        System.out.print("Masukkan Menit (0-59) : ");
                        m=sc.nextInt();
                        System.out.print("Masukkan Detik (0-59) : ");
                        s=sc.nextInt();

                        if (Jam_2511531001.isValid (h,m,s)) {
                                return new Jam_2511531001 (h,m,s);
                        } else {
                                System.out.println("[Error] Input Tidak
valid ! Silakan Ulangi.\n");
                        }
                        }
sc.close();
        }
```

Kode Program 2.3: Program JamDriver2_2511531001

Program dibawah ini adalah program driver yang meminta user atau pengguna untuk memasukkan nilai atribut jam, menit dan juga detik. Pada program ini j1 dideklarasikan jam pertama dan j2 dideklarasikan sebagai jam ke-2. Input akan dipecah lagi menjadi suatu function yang akan meminta user menginputkan masing masing nilai atribut yaitu jam, menit dan detik. Function tersebut juga akan melakukan perulangan dengan memanfaatkan method isValid yang akan meminta input ulang jika data tidak valid.

Setelah itu, program akan menampilkan data jam ke-1 dan data jam ke-2 dengan 2 format yaitu String dan Second. Setelah itu, program

ini akan menguji method compareTo yang akan menampilkan perbandingan antara kedua jam. Lalu, program ini akan menguji method durasiDetik(), plus(), nextSecond(), prevSecond().

Program ini akan menghasilkan output sebagai berikut :

```
===Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23) : 22
Masukkan Menit (0-59) : 50
Masukkan Detik (0-59) : 15
'
---Input Jam 2 ---
Masukkan Jam (0-23) : 17
Masukkan Menit (0-59) : 18
Masukkan Detik (0-59) : 10
'
--- Hasil Operasi ---
 Jam 1 (String) : 22:50:15
 Jam 2 (String) : 17:18:10
 Jam 1 dalam detik : 82215
 Jam 2 dalam detik : 62290
Status : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2) : -19925 detik
Jam 1 Detik Berikutnya : 22:1370:40
Jam 1 Detik Sebelumnya : 22:1370:36
Hasil J1 + J2 : 16:968:56

Program Selesai
```

Gambar 2.2 : Output Driver 2

2.3.Tugas Praktikum

Soal :

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

Gambar 2.3 : Soal Tugas Praktikum 1

Kode Jawaban :

a. Kode Program ADT “Mobil_2511531001”

```
public final class ADTmobil_2511531001 {
	private String nama;
	private String merk;
	private int tahun;
	private int cc;
	private long harga;

	public ADTmobil_2511531001 (String n, String m, int t, int cc, int h){
		this.nama = n; this.merk = m; this.tahun = t; this.cc =cc;
this.harga= h;
	}

	public ADTmobil_2511531001() {
		this.nama = "";
		this.tahun = 0;
		this.cc = 0;
		this.harga = 0;
		this.merk = "";
	}

	public String getNama() {return nama;}
	public String getMerk() {return merk;}
	public int getTahun () {return tahun;}
	public int getCC () {return cc;}
	public long getHarga () {return harga;}

	public void setNama (String n) {this.nama=n;}
	public void setMerk (String m) {this.merk=m;}
	public void setTahun (int t) {this.tahun=t;}
	public void setCC (int cc) {this.cc=cc;}
	public void setHarga (long h) {this.harga=h;}

	public void tambahMobil (String n, String m, int t, int cc, long h) {
		this.nama = n; this.merk = m; this.tahun = t; this.cc =cc;
this.harga= h;
	}
	public void hapusMobil() {
		this.nama = ""; this.tahun = 0; this.cc=0; this.harga=0;
this.merk="";
	}
	public void tampilkanInfo() {
		System.out.println("=== Informasi Mobil ===");
		System.out.println("Nama  : " + nama);
		System.out.println("Tahun : " + tahun);
		System.out.println("CC    : " + cc);
		System.out.println("Harga : Rp " + String.format("%,d", harga));
		System.out.println("Merk  : " + merk);
		System.out.println("======================");
	}
}
```

b. Kode Program Driver “Driver2_25115301001”

```
public class DriverADTmobil_2511531001 {
```

```
    public static void main (String [] args) {
            ADTmobil_2511531001 mobil1 = new
ADTmobil_2511531001("Avanza", "Toyota",2020, 1300,150000000);
            System.out.println("Menambahkan Mobil ke-1 dengan
konstruktor default");
            mobil1.tampilkanInfo();

            ADTmobil_2511531001 mobil2 = new
ADTmobil_2511531001();

    mobil2.tambahMobil("Civic","Honda",2021,1500,250000000);
            System.out.println("Menambah Mobil Ke-2 dengan method
tambahMobil");
            mobil2.tampilkanInfo();

            System.out.println("\nMengubah data mobil ke-2 dengan
setter:");
     mobil2.setTahun(2022);
     mobil2.setHarga(260000000);
     mobil2.tampilkanInfo();

     System.out.println("\nMengakses data menggunakan getter:");
     System.out.println("Nama Mobil: " + mobil2.getNama());
     System.out.println("Merk: " + mobil2.getMerk());
     System.out.println("Tahun: " + mobil2.getTahun());

     System.out.println("\nMenghapus mobil ke-1:");
     mobil1.hapusMobil();
     mobil1.tampilkanInfo();
    }
}
```

Output :

```
=== Informasi Mobil ===
Nama  : Avanza
Tahun : 2020
CC    : 1300
Harga : Rp 150,000,000
Merk  : Toyota
=======================
Menambah Mobil Ke-2 dengan method tambahMobil
=== Informasi Mobil ===
Nama  : Civic
Tahun : 2021
CC    : 1500
Harga : Rp 250,000,000
Merk  : Honda
=======================

Mengubah data mobil ke-2 dengan setter:
=== Informasi Mobil ===
Nama  : Civic
Tahun : 2022
CC    : 1500
Harga : Rp 260,000,000
Merk  : Honda
=======================

Mengakses data menggunakan getter:
Nama Mobil: Civic
Merk: Honda
Tahun: 2022

Menghapus mobil ke-1:
=== Informasi Mobil ===
Nama  :
Tahun : 0
CC    : 0
Harga : Rp 0
Merk  :
=======================
```

Penjelasan : Program tugas ini menggunakan konsep getter dan setter untuk memasukkan dan mengubah data mobil. Selain menggunakan setter, data mobil juga bisa diinputkan dengan method TambahMobil dan menghapus data dengan hapusMobil. Setelah itu, kita dapat menggunakan method tampilkanInfo yang bisa yang menampilkan data dari mobil yang telah dimasukkan.

BAB III
KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat dikumpulkan bahwa Abstract Data Type (ADT) merupakan model yang mendefinisikan suatu tipe data melalui sekumpulan nilai dan operasi yang dapat dilakukan tanpa mengungkapkan detail implementasi internalnya. Implementasi pada praktikum ini menghadirkan 2 program yang driver yang akan menjalankan primitive yang sudah didefinisikan

pada program ADT. Kedua program driver memerlukan deklarasi ke-2 jam (driver pertama adalah a&b, driver ke-2 adalah j1 & j2).

Dalam konteks pemrograman Java, ADT memanfaatkan interface, class, private access modifie untuk pemisahan yang jelas antara spesifikasi dan implementasi. Penggunaan atribut internal yang dideklarasikan sebagai private sehingga hanya dapat diakses melalui method public yang terkontrol seperti getter dan setter. Pendekatan ini memungkinkan validasi data terpusat, mencegah korupsi state objek serta memberikan fleksibilitas untuk mengubah representasi internal.

DAFTAR PUSTAKA

[1Datta.S, "Abstract Data Type (ADT)," Baeldung, [2024] https://www.baeldung.com/cs/adt [Diakses 3 Apr 2026].

[2] Liem.Inggriani “Diktat Struktur Data” [2008] Hal 7- 15.